Improving Dynamic Programming for Knapsack

Date: 2026-11-06

Abstract

The standard dynamic program for the 0-1 knapsack problem (DP-1) always takes $\Theta(nc)$ time, where c is the capacity. We implement two improvements from Sections 3.4 and 3.5 of Kellerer, Pferschy and Pisinger: DP-with-Lists keeps only undominated (weight, profit) states, and DP-with-Upper-Bounds also drops states whose LP upper bound can not beat the best solution found so far. For the bonus part we implement the $O(n + w^3)$ method of Polak, Rohwedder and Węgrzycki, which repairs a greedy solution with a few items found by concave convolutions and the SMAWK algorithm. All methods agree with brute force and an independent DP on 3,312 test cases. In four experiments on five types of generated data, DP-with-Lists beats DP-1 only when the capacity is large, for example about 250 times at $c = 2.5 \times 10^8$. DP-with-Upper-Bounds is up to 3,000 times faster than DP-1 on uncorrelated and weakly correlated data, but it can be slower than DP-1 on subset-sum data, and its time on strongly correlated data changes a lot from input to input. The bonus method solves 10^6 items with weights up to about 100 in less than 90 ms on every type, and its time grows like w^3 as expected.

Contents

1 Introduction **3**

2 Problem Description **3**

3 Algorithm Specification **3**

3.1 DP-1 . . . 3

3.2 DP-with-Lists . . . 4

3.3 DP-with-Upper-Bounds . . . 4

3.4 Small-Weight Method (Bonus) . . . 5

3.5 Correctness . . . 6

4 Testing Results **7**

4.1 Method . . . 7

4.2 Correctness Tests . . . 7

4.3 Test Data . . . 8

4.4 Timing Method . . . 8

4.5 Experiment 1: Number of Items . . . 8

4.6 Experiment 2: Weight Range . . . 10

4.7 Experiment 3: Many Small Items (Bonus) . . . 11

4.8 Experiment 4: Largest Weight (Bonus) . . . 12

5 Analysis and Comments **13**

5.1 Time Complexity . . . 13

5.2 Space Complexity . . . 14

5.3 Discussion of Test Results . . . 14

5.4 Limitations and Improvements . . . 16

6 Conclusion **17**

7 References **17**

8 Source Code Appendix **17**

1 Introduction

The knapsack problem asks which items to pack when the bag can not hold all of them. It appears whenever a budget, a disk or a truck has a fixed size and each choice has a cost and a value. The textbook method is a dynamic program (DP) that fills a table with one row per item and one column per capacity. It is simple and always correct, but it always fills the whole table, even when the answer is easy to see.

In this project we follow Sections 3.4 and 3.5 of the book by Kellerer, Pferschy and Pisinger [1]. Section 3.4 keeps only the useful entries of each row in a sorted list, and Section 3.5 also throws away entries that can not lead to a better answer, using an upper bound as in branch and bound. For the bonus part we implement the $O(n + w^3)$ algorithm of Polak, Rohwedder and Węgrzycki [2], which does not depend on the capacity at all. We generate five kinds of test data, check that all methods give the same optimal value, and measure how their running times change with the number of items, the size of the weights and the type of data.

2 Problem Description

We are given n items and a knapsack with capacity c. Item i has a positive integer weight w_i and a non-negative integer profit p_i. We want a set of items whose total weight is at most c and whose total profit is as large as possible. We only need this largest profit, not the set itself.

The standard DP from class is called DP-1 in [1]. Let $z_j(d)$ be the best profit using only the first j items and capacity d. Then $z_0(d) = 0$, and for $j \geq 1$

$$z_j(d) = \max\{z_{j-1}(d),\ z_{j-1}(d - w_j) + p_j\},$$

where the second choice is only allowed when $d \geq w_j$. The answer is $z_n(c)$. Filling the table takes $\Theta(nc)$ time in every case. This is the main problem: c is a number in the input, not the input size. With 1,000 items of weight up to 10^6, the capacity is about 2.5×10^8, so the table has 2.5×10^{11} entries. On our machine DP-1 fills about 3.5 entries per nanosecond, so this would take more than a minute and 2 GB of memory for one row. In contrast, DP-with-Upper-Bounds solves our uncorrelated and weakly correlated inputs of this size in less than 2 ms.

Our programs read `n c` on the first line and then `w p` for each item, and print the optimal profit. Weights and profits are at most 10^9.

3 Algorithm Specification

3.1 DP-1

We implement DP-1 with one array `z[0..c]` instead of the whole table. For each item we update `z[d]` for d from c down to w_j. Going down is the key point: when we compute `z[d]`, the entry `z[d - w]` has not been changed yet, so it still holds $z_{j-1}(d - w_j)$. This gives the same values as

the two-loop table in class, with $O(c)$ memory instead of $O(nc)$. We use it as the baseline for all experiments.

3.2 DP-with-Lists

Most entries of a DP row are not interesting. If $z_j(5) = z_j(6) = z_j(7)$, the entries at 6 and 7 add nothing, because capacity 5 already gives the same profit with less weight. Section 3.4 of [1] therefore stores each row as a list of states (w, p): a set of the first j items with weight sum w and profit sum p. A state (w, p) dominates (w', p') if it is not heavier and not less profitable, and it is better in at least one of the two [1]. A dominated state can never lead to an optimal solution, because the same items added to the dominating state give a solution that is at least as good, so we delete it [1].

We keep every list sorted by weight. After deleting the dominated states, the profits also increase along the list, so the last state always has the largest profit. One step from list L_{j-1} to L_j works as follows:

1. **Add the item.** Add (w_j, p_j) to every state of L_{j-1}. We stop at the first state whose new weight is more than c; since the list is sorted, all later states are too heavy as well.
2. **Merge.** Merge the old list and the new list like in merge sort, always taking the lighter state first. A state is kept only if its profit is larger than the profit of the last kept state. If it has the same weight as the last kept state, it replaces that state.

The answer is the profit of the last state in L_n. For the example of Fig. 3.7 in [1] with $c = 9$ (the figure prints each state with the profit first), the first item $(2, 6)$ gives $L_1 = ((0, 0), (2, 6))$. The second item $(3, 5)$ gives the new states $(3, 5)$ and $(5, 11)$. When we merge, $(3, 5)$ is dropped, because $(2, 6)$ is lighter and more profitable, so $L_2 = ((0, 0), (2, 6), (5, 11))$.

The worst case is still $O(nc)$, since a list can hold up to $c + 1$ states. However, if every list has at most B states, the time is $O(nB)$ [1], and B can be much smaller than c when the weights are large.

3.3 DP-with-Upper-Bounds

A state of the list DP is also a node of a branch-and-bound tree: it fixes which of the first j items are packed. Section 3.5 of [1] uses this to add a second kind of pruning. We keep an incumbent z, the profit of the best real solution found so far. For every state we compute an upper bound $U(w, p)$ on any solution that grows from it. If $U(w, p) \leq z$, the state can not give anything better than z, so we drop it.

As in [1], the upper bound is the LP relaxation: we fill the remaining room $c - w$ with items $j + 1, \ldots, n$ and allow the last item to be packed in part. To make this fast, we first sort the items

by efficiency p_i/w_i from high to low. Then the best fractional filling simply takes the next items in order. With prefix sums $W_k = \sum_{i \le k} w_i$ and $P_k = \sum_{i \le k} p_i$, we use binary search to find the largest k with $W_k - W_j \le c - w$, and

$$U(w, p) = p + (P_k - P_j) + \lfloor (c - w - (W_k - W_j))\, p_{k+1}/w_{k+1} \rfloor.$$

We may round down because all profits are integers. The full step for item j is:

1. **Grow.** Build L_j from L_{j-1} exactly as in DP-with-Lists.
2. **Update the incumbent.** Every state is a real solution, so we set z to the larger of z and the profit of the last state.
3. **Bound.** Remove every state with $U(w, p) \le z$.

Before the first item, z is the profit of the greedy solution, which goes through the sorted items and packs every item that still fits. The loop stops early when the list becomes empty, and the answer is z. We compare efficiencies by $p_a w_b > p_b w_a$ with 128-bit integers, so no floating-point error can change the order.

Sorting changes the order in which DP-with-Lists sees the items, and the order alone may change the number of states. So we also run DP-with-Lists on the sorted items (called "lists, sorted" below). This lets us see how much of the speed-up comes from the bound and how much from the order.

3.4 Small-Weight Method (Bonus)

Let $w = \max_i w_i$ be the largest weight. The method of [2] runs in $O(n + w^3)$ time, which does not depend on c. It starts from a greedy solution and repairs it by changing only a few items. We first drop items that are heavier than c, and if all remaining items fit together, the answer is their total profit.

Greedy prefix. We order the items by efficiency and take them one by one until the next item does not fit. This set G is called the maximal prefix solution [2]. Let $\Delta = c - \sum_{i \in G} w_i$ be the room left; it is less than w, because the next item did not fit. We do not sort all items to find G. Instead we use quickselect with a random pivot: if the items more efficient than the pivot already weigh more than the room, G ends among them; otherwise we take all of them and continue with the rest. Each round throws away part of the items, so the expected time is $O(n)$.

Proximity. The key fact from [2] is that some optimal solution differs from G in at most $2w$ items. The idea of the proof is an exchange argument: if the two solutions differed in more items, we could find a group of items removed from G and a group of items added to G with the same total weight. The removed items are at least as efficient as the added ones, so swapping the groups back gives an optimal solution that is closer to G. Since each item weighs at most w, the removed

items weigh at most $2w^2$ in total. So we only need to try "remove weight exactly k from G, then add weight at most $k + \Delta$ from the other items" for $k = 0, \ldots, 2w^2$.

Best removal and best addition. For each $k \leq 2w^2$ we compute the smallest profit we lose by removing items of G with total weight exactly k. For each $y \leq 2w^2 + \Delta$ we compute the largest profit we gain by adding other items with total weight exactly y, and then take a running maximum to get "at most y". Both are knapsack problems, but with a small capacity $T = O(w^2)$. We solve them one weight class at a time. Among items of the same weight h, using j of them always means using the j best ones, so their value is a prefix sum b_j of the sorted profits. Since the profits go down, b is concave: each new item adds no more than the one before. Adding this class to the current array f is a max-plus convolution

$$f'(r + ih) = \max_j \{f(r + jh) + b_{i-j}\},$$

done separately for each remainder $r < h$. Because b is concave, the matrix of all these sums is Monge, so its row maxima can be found in linear time by the SMAWK algorithm [3], [2]. Entries outside the real range of b get a very steep penalty, so the extended b stays concave and SMAWK can still be used. For weight h we keep only the best $\lfloor T/h \rfloor$ items, since more would not fit anyway.

Combine. The answer is the profit of G plus the best value of "gain with weight at most $k + \Delta$ minus loss with weight exactly k" over all k. Every such choice weighs at most c, so it is a real solution.

3.5 Correctness

DP-1 is the recurrence above, and the downward loop keeps the old row values that the recurrence needs. DP-with-Lists keeps exactly the undominated states of each row. Because a dominated state never leads to an optimal solution [1], for every capacity d the list L_j still holds a state with weight at most d and profit $z_j(d)$. So the last state of L_n has profit $z_n(c)$.

DP-with-Upper-Bounds only removes more states. A state is removed only when its LP bound is at most z. The LP relaxation allows more than the real problem, so no real completion of that state beats z. Since z is always the profit of a real solution, the final z is optimal.

For the small-weight method, the proximity result of [2] shows that one optimal solution is "remove weight $k \leq 2w^2$ from G, add weight at most $k + \Delta$". Our two arrays give the best removal and the best addition for every weight, so the best combination is at least as good as that solution. It is also a real solution, so it is optimal. The convolution is correct because the best j items of one weight are always the j most profitable ones, and SMAWK returns exact row maxima of a Monge matrix [3].

4 Testing Results

4.1 Method

All tests were run on macOS 26.6 with an Apple M5 (10 cores), Apple Clang 21.0.0 and Python 3.9. We compile with `-O2`, and also with AddressSanitizer and UndefinedBehaviorSanitizer to find memory errors.

We check the C++ programs against two independent oracles written in Python. For $n \leq 12$ we try all subsets. For larger inputs with small capacity we use a separate Python version of DP-1. When an input is too large for both, all methods must print the same value. The program `knapsack all` runs every method on the same input, so one test checks all of them at once.

4.2 Correctness Tests

Test group (count)	Result	Purpose
Hand-made cases (11)	PASS	No items, capacity 0, every item too heavy, all items fit, one item fills the bag, equal efficiencies, zero profits, greedy is not optimal, 300 copies of one item, and the example of Fig. 3.7 in [1].
Random small inputs (3,000)	PASS	$n \leq 12$ and weights up to 100, with capacity from 0 to above the total weight; compared with all subsets. Small ranges give many ties.
Many small items (200)	PASS	20 to 200 items with weights 1 to 8, so each weight class of the bonus method holds many items; compared with Python DP.
Generated inputs (90)	PASS	All five data types, n from 50 to 200, R from 10 to 1,000; compared with Python DP when $nc \leq 3 \times 10^5$, otherwise all methods must agree. The generator must repeat itself for the same seed.
Large inputs (6)	PASS	Capacity about 2.5×10^9, which DP-1 and the bonus method refuse, so the list methods must agree; and 20,000 items with weights up to 10, where all five methods must agree.
Bad inputs (5)	PASS	Missing items, weight 0, negative capacity, negative profit and empty input must be rejected.

All 3,312 cases pass. The sanitizer build runs a smaller set of 586 cases and reports no error. In addition, the timing script below checks that all methods print the same value on every input it measures.

4.3 Test Data

Our generator `code/gen.cpp` makes five types of data, which are common test types in the knapsack literature [4]. Here R is the weight range and $U(a, b)$ is a random integer between a and b:

1. **uncorrelated**: $w_i = U(1, R)$ and $p_i = U(1, R)$, chosen independently.
2. **weak** (weakly correlated): $w_i = U(1, R)$ and $p_i = U(w_i - R/10,\ w_i + R/10)$, at least 1.
3. **strong** (strongly correlated): $w_i = U(1, R)$ and $p_i = w_i + R/10$.
4. **inverse** (inverse strongly correlated): $p_i = U(1, R)$ and $w_i = p_i + R/10$.
5. **subset** (subset sum): $w_i = U(1, R)$ and $p_i = w_i$.

The capacity is half of the total weight, so about half of the items fit. In the correlated types the profit of a set depends mostly on its total weight. For example, in the strong type a set with weight W and k items has profit $W + kR/10$. So many different sets have almost the same profit, and it is hard to prove that one of them is the best. We will see that this matters a lot for the methods with bounds.

4.4 Timing Method

Our performance test program `code/bench.cpp` reads one input file and runs one method on it once as warmup and then 10 more times. It times only the method, not reading the input, and reports the median of the 10 runs. Every setting uses three inputs (seeds 1, 2 and 3). The tables give the median of the three per-input medians in milliseconds, and the error bars in the figures go from the fastest to the slowest of the three inputs. To keep the total time near one hour, we stop running a method on larger inputs of a type once its median passes 500 ms; such cells are marked "–" . DP-1 also refuses inputs with $c > 2 \times 10^8$, because its array would need more than 1.6 GB.

The "work" of a run is the number of table entries for DP-1, the total number of list states for the list methods, and the number of convolution entries for the small-weight method. It does not depend on the machine, so it helps to explain the times.

4.5 Experiment 1: Number of Items

We fix $R = 1000$ and use $n = 250, 500, 1000, 2000, 4000$. The capacity is half the total weight, about $250n$, so it grows with n and the DP table grows like n^2. The small-weight method is not run here, because $w \approx 1000$ makes $w^3 = 10^9$ too large. The first table gives the median time in ms for $n = 1000$, and the second table gives the work of the same runs.

Type	DP-1	Lists	Lists, sorted	Bounds
uncorrelated	74.7	211	268	0.0718
weak	70.9	480	483	1.07
strong	68.0	995	240	91.9
inverse	84.2	894	476	14.1
subset	67.8	332	482	943

Type	DP-1 cells	Lists states	Lists, sorted	Bounds states
uncorrelated	2.5×10^8	3.3×10^7	4.4×10^7	1.9×10^3
weak	2.6×10^8	1.0×10^8	1.3×10^8	5.3×10^4
strong	2.6×10^8	1.9×10^8	1.4×10^8	6.4×10^6
inverse	3.1×10^8	2.0×10^8	2.3×10^8	8.1×10^5
subset	2.6×10^8	1.9×10^8	1.9×10^8	6.2×10^7

At $n = 4000$ DP-1 takes 1,130 to 1,380 ms on every type, while DP-with-Upper-Bounds takes 0.97 ms (uncorrelated), 0.74 ms (weak), 73 ms (inverse) and 500 ms (strong).

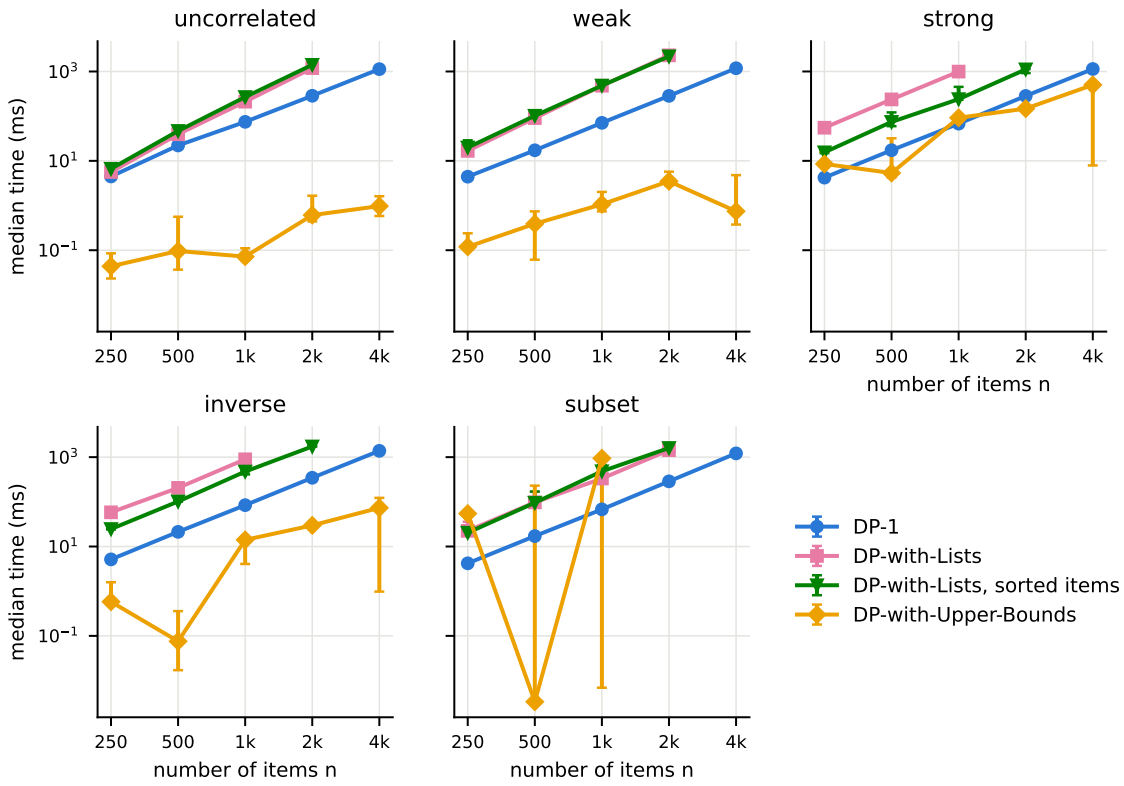


Figure 1: Experiment 1: median time against the number of items, R = 1000

4.6 Experiment 2: Weight Range

We fix $n = 1000$ and use $R = 10^2, 10^3, \ldots, 10^6$, so the capacity grows from about 2.5×10^4 to 2.5×10^8 while the number of items stays the same. The table shows the median time in ms for the uncorrelated type, together with the total number of states of DP-with-Lists.

R	Capacity c	DP-1	Lists	Bounds	Lists states
10^2	2.5×10^4	6.56	53.0	0.093	1.3×10^7
10^3	2.5×10^5	70.6	204	0.065	3.3×10^7
10^4	2.6×10^6	830	277	0.261	4.2×10^7
10^5	2.5×10^7	–	276	0.138	4.2×10^7
10^6	2.5×10^8	–	263	0.161	4.0×10^7

For the other types, DP-with-Upper-Bounds at $R = 10^6$ takes 1.29 ms (weak) and 2,160 ms (inverse); for strong it already takes 2,660 ms at $R = 10^5$.

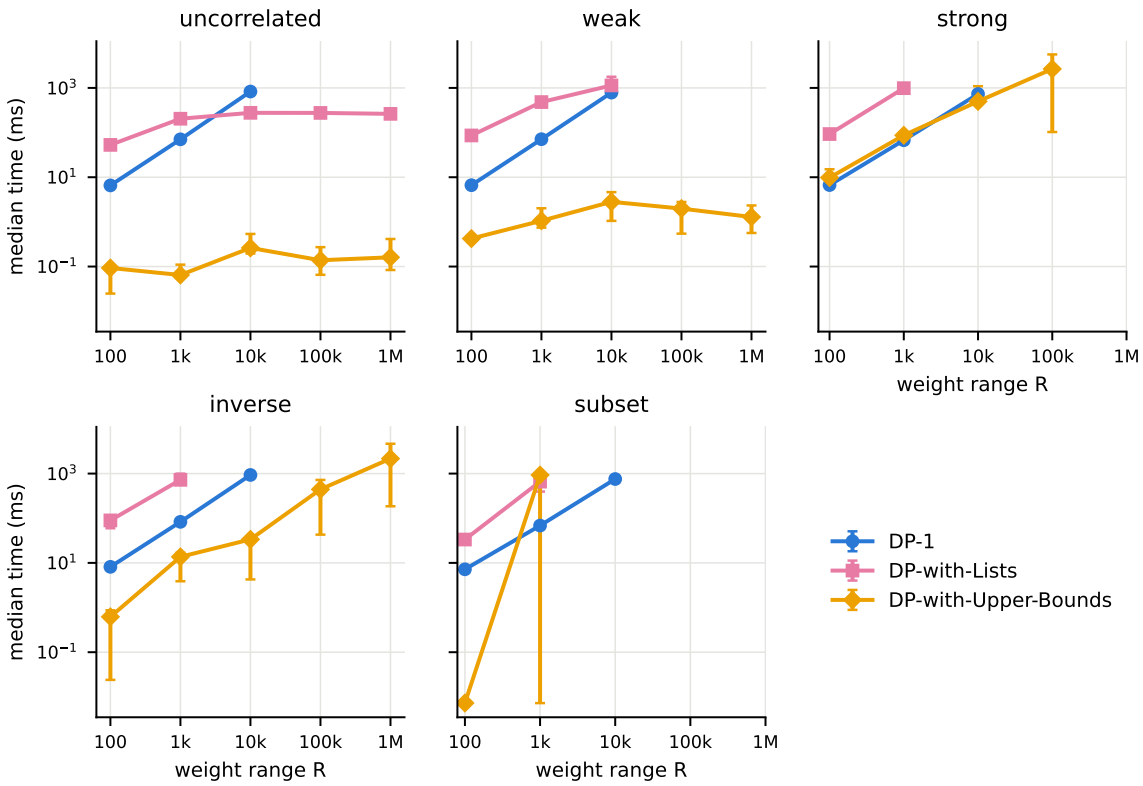


Figure 2: Experiment 2: median time against the weight range, n = 1000

4.7 Experiment 3: Many Small Items (Bonus)

For the bonus method we fix the weight range $R = 100$ (so $w \leq 100$, and $w \leq 110$ for the inverse type) and use $n = 10^3, 10^4, 10^5, 10^6$. The capacity grows from about 2.5×10^4 to 2.5×10^7. DP-1 is only run up to $n = 10^4$, where it already takes 680 to 840 ms. The table compares the two other methods (median time in ms).

Type	Bounds, $n = 10^4$	Small, $n = 10^4$	Bounds, $n = 10^6$	Small, $n = 10^6$
uncorrelated	0.384	70.7	57.3	88.1
weak	0.376	71.9	50.3	86.1
strong	205	35.2	–	48.8
inverse	13.2	43.1	41.5	58.3
subset	0.091	65.6	11.8	72.9

For the strong type, DP-with-Upper-Bounds takes 1,470 ms at $n = 10^5$ and is not run at $n = 10^6$.

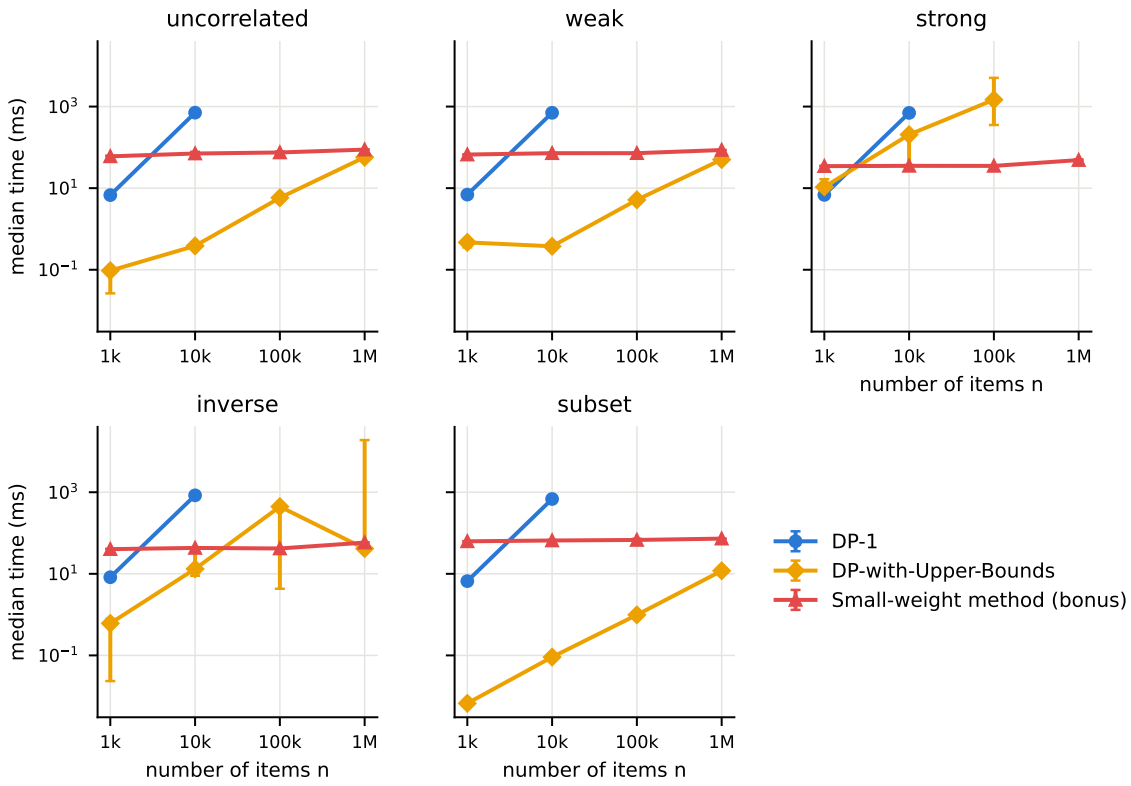


Figure 3: Experiment 3: median time against the number of items, R = 100

4.8 Experiment 4: Largest Weight (Bonus)

To see the w^3 term, we fix $n = 10^4$ and double the weight range: $R = 10, 20, 40, 80, 160, 320$. The table gives the median time in ms. DP-1 and the small-weight method are shown for the uncorrelated type and DP-with-Upper-Bounds for the strong type, where it is slowest. The last column is the work of the small-weight method on uncorrelated data.

R	DP-1, uncorr.	Bounds, strong	Small, uncorr.	Small, strong	Small work, uncorr.
10	74.9	26.4	0.245	0.171	1.4×10^4
20	144	15.1	0.982	0.534	4.2×10^4
40	278	96.6	6.19	2.81	2.6×10^5
80	558	116	38.2	19.4	2.1×10^6
160	–	161	255	130	1.6×10^7
320	–	139	1,850	963	1.3×10^8

On the other four types DP-with-Upper-Bounds needs at most 18 ms for every R, because the greedy start is optimal or almost optimal there.

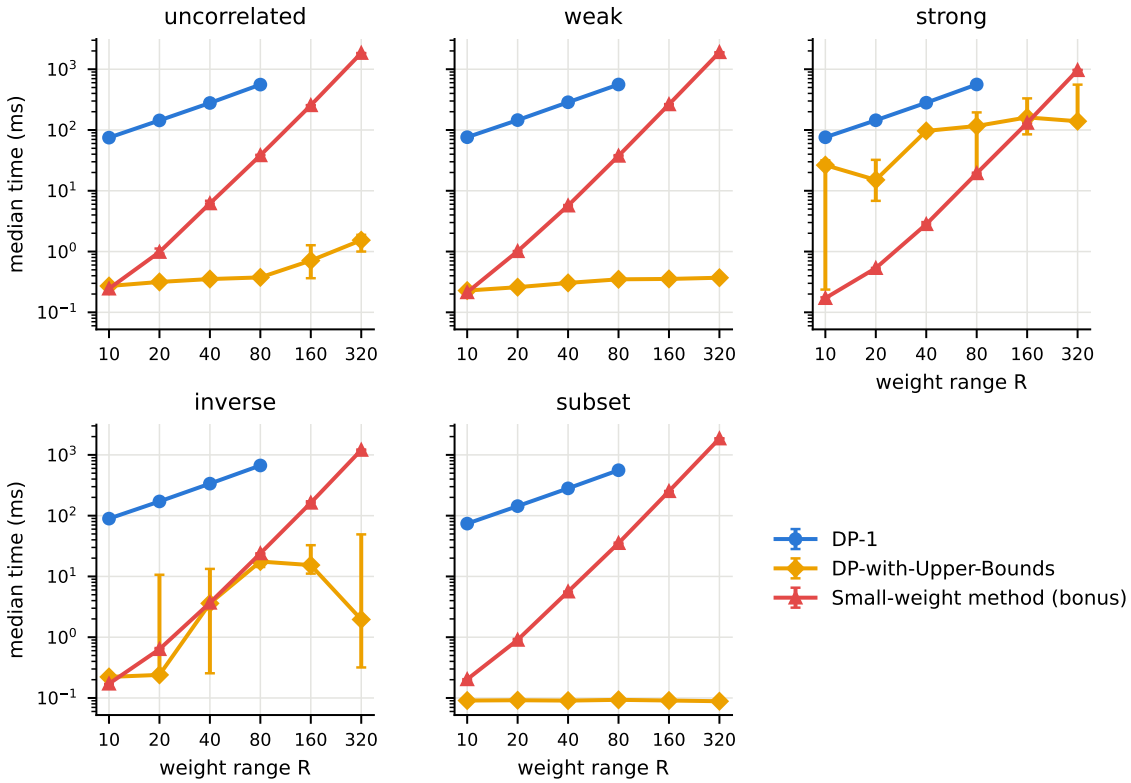


Figure 4: Experiment 4: median time against the weight range, n = 10,000

5 Analysis and Comments

5.1 Time Complexity

DP-1. Each item updates at most $c + 1$ entries in constant time, so the time is $\Theta(nc)$ in every case.

DP-with-Lists. Adding the item and merging are both linear in the list length. If B_j is the length of L_j and $S = \sum_j B_j$ is the total number of states, the time is $O(n + S)$. Since $B_j \leq c + 1$, this is $O(nc)$ in the worst case, as stated in [1]. The best case is much better and can be as low as $O(n)$ [1]. For example, if every item has weight c, each list holds only $(0, 0)$ and the best single item. In general the states of one list have different weights and different profits, so B_j is also at most 2^j and at most the number of different profit sums. So B_j can be much smaller than c when c is large but the items are few.

DP-with-Upper-Bounds. Sorting takes $O(n \log n)$. Each state needs one bound, and our bound uses a binary search, so it costs $O(\log n)$. The total time is $O(n \log n + S' \log n)$, where $S' \leq S$ is the number of states that survive. In the worst case nothing is pruned and this is $O(nc \log n)$. The

book evaluates each bound in $O(n)$ and gets $O(n^2c)$ [1]; the prefix sums are what make our bound cheaper. As in branch and bound, the worst case says little here: when the bound is tight, S' is tiny.

Small-weight method. Finding the greedy prefix takes expected $O(n)$ time. Putting the items into w weight classes and picking the best $\lfloor T/h \rfloor$ items of each class takes $O(n + w)$, and sorting the picked items takes $O(\sum_h (T/h) \log T) = O(w^2 \log^2 w)$. For one weight h we call SMAWK once for each remainder $r < h$, on a matrix with about T/h rows, and SMAWK is linear in the matrix size [3]. So one weight class costs $O(T + h)$, and all w classes cost $O(w(T + w)) = O(w^3)$ because $T < 2w^2 + w$. We do this twice, once for removing and once for adding. The total expected time is $O(n + w^3)$, which matches [2] except that [2] finds the prefix in deterministic $O(n)$ time with a linear-time median.

5.2 Space Complexity

DP-1 needs $O(c)$ for one row. The two list methods keep only the current list and the next one, so they need $O(n + B)$ space with $B = \max_j B_j \leq c + 1$. The small-weight method needs $O(n)$ for the items and $O(w^2)$ for the arrays of length T, so $O(n + w^2)$ in total. It is the only method whose memory does not grow with c.

5.3 Discussion of Test Results

DP-1 does the same work on every input. In Experiment 1 its time grows by a factor of about 4 each time n doubles (the slope on the log scale is 2.0), because c also doubles. In Experiment 2 it grows by a factor of 10 each time R grows by 10. The five types give almost the same time, about 0.28 ns per table entry. This matches $\Theta(nc)$: the data itself does not matter at all, only n and c.

Fewer states does not always mean faster. In Experiment 1, DP-with-Lists keeps only 13% of the DP-1 entries on uncorrelated data and 40% to 75% on the other types, yet it is 3 to 15 times slower. The reason is the cost of one unit of work. The DP-1 loop is very simple, so the compiler turns it into vector instructions that update two entries at once (we checked the generated code), and one entry costs about 0.3 ns. One list state costs 2 to 7 ns: it is written twice (once when the item is added and once when the lists are merged), it takes 16 bytes, and the merge has a branch that is hard to predict. So the lists only pay off when they hold far fewer states than DP-1 has entries, roughly one state for every 10 to 20 entries.

The lists win when the capacity is large. Experiment 2 shows exactly this case. With uncorrelated data the total number of states stops growing after $R = 10^4$ and stays near 4×10^7, about 40,000 states per list, while c grows to 2.5×10^8. So the time stays near 270 ms. By Lemma 3.4.3 of [1] the time is $O(nB)$, and here B depends on how many useful (weight, profit) pairs exist, not on c. DP-1 would need about 70 s at $R = 10^6$, so the lists are about 250 times faster there. The

crossing point lies between $R = 10^3$ and $R = 10^4$. For the correlated types, however, the number of states keeps growing with R, and the lists are slow already at $R = 10^3$.

The upper bound is what makes the big difference. Sorting the items alone changes the number of states by at most about one third (the "Lists, sorted" column). It makes the strong and inverse types 2 to 4 times faster, mainly because each state becomes cheaper to process, but it is still much slower than DP-1. Adding the bound cuts the states from 4.4×10^7 to about 2×10^3 on uncorrelated data, and the time from 268 ms to 0.07 ms. The bound works well when the efficiencies of the items are spread out. Then the LP bound is close to the real optimum, and soon after an item is processed, the states that skip a very efficient item or take a very inefficient one have bounds below z. Only states that differ in items with efficiency close to the last item that fits survive. On uncorrelated and weak data, DP-with-Upper-Bounds is 16 to 3,000 times faster than DP-1 in Experiments 1 and 2, and the gap tends to grow with n and R, because the time of DP-with-Upper-Bounds changes little.

Correlated types make the bound weak. In the strong and inverse types, many items near the last item that fits have almost the same efficiency, so many states have bounds just above z and can not be removed. In Experiment 1 DP-with-Upper-Bounds is about as fast as DP-1 on strong data for $n \leq 1000$ and only about 2 times faster at $n = 4000$ (500 against 1,136 ms). Its time also grows with R in Experiment 2, and it varies a lot between seeds: one strong input with $n = 4000$ took only 7.9 ms, the other two about 500 ms.

The subset type is all or nothing. Here every item has efficiency 1, so the LP bound of every state is simply c as long as the remaining items can fill the room. No state can be removed until some state reaches weight exactly c; then $z = c$ and everything is removed at once. When our greedy start already fills the knapsack exactly, the method stops after 3 states. Otherwise it must grow the lists until an exact fill appears. In Experiment 1, two of the three inputs with $n = 500$ stopped after 3 states, but only one input with $n = 1000$ did. The other two needed about 6×10^7 states and 943 ms, which is 14 times slower than DP-1. So the median jumps up and down, and the error bars cover more than four orders of magnitude.

The small-weight method does not care about n, c or the type. In Experiment 3, n grows 1,000 times and c from 2.5×10^4 to 2.5×10^7, but its time only grows from 35 to 49 ms (strong) or from 60 to 88 ms (uncorrelated). Its work is about 2×10^6 to 5×10^6 entries in every case, which is set by $w = 100$, and the $O(n)$ part adds only about 15 ms at $n = 10^6$. The strong and inverse types need only half to two thirds of the work of the others. In these types the efficiency depends on the weight, so the greedy prefix holds mostly items of some weights and the other items have the remaining weights. Each of the two convolutions then sees only part of the weight classes.

So which method is best depends on the data. When the greedy start is already optimal and the bound proves it at once (most of the uncorrelated, weak and subset inputs with many small items), DP-with-Upper-Bounds only sorts the items and is faster than the small-weight method, for example 50 ms against 86 ms for weak data with $n = 10^6$. For the strong type, however, it takes

205 ms at $n = 10^4$ and 1,470 ms at $n = 10^5$, while the small-weight method takes 35 ms in both cases, about 40 times less. The small-weight method is the only one whose time we can predict from n and w alone.

The small-weight method grows like w^3. In Experiment 4, each time R doubles, its work grows by a factor of 7.9 on uncorrelated data, very close to $2^3 = 8$, and the slope of its time on the log scale rises to 2.9 for large R. For small R the slope is lower, because the fixed costs (putting the 10^4 items into weight classes and choosing the best ones) still matter. The time per entry is 14 to 24 ns, much more than one DP-1 entry, since each SMAWK step calls the value function several times and works with 128-bit numbers. Still, for $R \leq 80$ it is 15 to 300 times faster than DP-1, whose time grows like R because c does. For the strong type it beats DP-with-Upper-Bounds up to $R = 160$ (130 against 161 ms), but at $R = 320$ it is 7 times slower. So the method is best when the weights are small and the data is hard for the bound, which is the setting it was designed for [2].

The bound method can be very fast or very slow on the same kind of data. The clearest example is the inverse type with $n = 10^6$ in Experiment 3. Two of the three inputs took 41 ms, because the greedy start was optimal and the bound removed every state at once. The third input needed 5×10^8 states and 19 s. The small-weight method took 58 ms on all three. This is the main practical difference between the methods: the time of DP-1 and of the small-weight method follows from n, c and w, while the time of the bound methods depends on how good the bound is on that particular input.

5.4 Limitations and Improvements

1. The bound methods depend on a good first incumbent z. Our greedy start is weak on subset and strong data, so the bound can not remove states until a very good solution shows up, and the time changes a lot from input to input. A better start, for example trying to fill the room left by the greedy solution with a small DP over the items near the last one that fits, would help exactly these cases.
2. The small-weight method always works on arrays of length $2w^2$ and does one convolution per weight, so its time grows like w^3 even when the input is easy. A sharper proximity bound would shorten the arrays, and weight classes with very few items could be added by plain DP instead of SMAWK.
3. All timings come from one machine and three inputs per setting. Some results, especially for the subset and strong types, change a lot between seeds, and the fast vector instructions that help DP-1 are not available on every machine, so the crossing points between the methods may move on other hardware.

6 Conclusion

We implemented four exact methods for the knapsack problem and compared them on the same inputs. DP-1 is simple and its time follows nc exactly, whatever the data. Keeping only undominated states (Section 3.4 of [1]) makes the time depend on the number of useful states instead of on c, which pays off when the capacity is very large; however, each state costs 7 to 25 times more than a DP-1 entry, so for moderate capacities it is slower than DP-1. Adding the LP upper bound (Section 3.5 of [1]) gives by far the largest speed-up on uncorrelated and weakly correlated data, but it relies on a tight bound and a good first solution, and it can fail badly on subset-sum and strongly correlated inputs. The $O(n + w^3)$ method of [2] avoids this: its time depends only on n and the largest weight w, so it is the most reliable choice when the weights are small, and our measurements confirm the w^3 growth. In short, the best method depends on the data, and the experiments show both when each improvement works and why.

7 References

[1] H. Kellerer, U. Pferschy, and D. Pisinger, *Knapsack Problems*. Berlin, Heidelberg, Germany: Springer, 2004, ch. 3, sec. 3.4–3.5, pp. 50–54. doi:10.1007/978-3-540-24777-7.

[2] A. Polak, L. Rohwedder, and K. Węgrzycki, "Knapsack and Subset Sum with Small Items," in *48th International Colloquium on Automata, Languages, and Programming (ICALP 2021)*, LIPIcs, vol. 198. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021, pp. 106:1–106:19. doi:10.4230/LIPIcs.ICALP.2021.106.

[3] A. Aggarwal, M. M. Klawe, S. Moran, P. Shor, and R. Wilber, "Geometric applications of a matrix-searching algorithm," *Algorithmica*, vol. 2, no. 1–4, pp. 195–208, 1987. doi:10.1007/BF01840359.

[4] D. Pisinger, "Where are the hard knapsack problems?," *Computers & Operations Research*, vol. 32, no. 9, pp. 2271–2284, 2005. doi:10.1016/j.cor.2004.03.002.

8 Source Code Appendix

The source code is in the `code/` folder of the submission. `code/knapsack.cpp` contains the four algorithms: `dp1`, `dp_lists`, `dp_bounds` and `small_items`, with `merge_lists`, `smawk` and `concave_convolution` as helpers; `code/knapsack.h` declares them. `code/main.cpp` solves one instance, `code/bench.cpp` is the performance test program, and `code/gen.cpp` is the test data generator. The `README.md` shows how to compile and run them.